

منصة التعليم الجامعي - مشروع التخرج

وصف المشروع

منصة ويب تربط بين الطلاب والأكاديميين والأساتذة للإدارة الأكاديمية والتواصل الفعال.

المتطلبات التقنية

- **Frontend:** React.js with Tailwind CSS
- **Backend:** Express.js
- **Database:** SQLite
- **Authentication:** JWT Tokens
- **API:** RESTful API

هيكل المشروع

Code

```
university-platform/
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   │   ├── authController.js
│   │   │   ├── studentController.js
│   │   │   ├── professorController.js
│   │   │   └── adminController.js
│   │   ├── models/
│   │   │   ├── User.js
│   │   │   ├── Course.js
│   │   │   ├── Assignment.js
│   │   │   └── Grade.js
│   │   ├── routes/
│   │   │   ├── auth.js
│   │   │   ├── students.js
│   │   │   ├── professors.js
│   │   │   └── admin.js
│   │   ├── middleware/
│   │   │   ├── auth.js
│   │   │   └── roleCheck.js
│   │   ├── config/
│   │   │   └── database.js
│   │   └── app.js
│   └── package.json
└── frontend/
```

قاعدة البيانات (Database Schema)

SQL

```
CREATE TABLE student_courses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  student_id INTEGER,  
  course_id INTEGER,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (student_id) REFERENCES users(id),  
  FOREIGN KEY (course_id) REFERENCES courses(id),  
  UNIQUE(student_id, course_id)  
);
```

جدول المهام (Assignments)

SQL

```
CREATE TABLE assignments (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  course_id INTEGER,  
  title TEXT NOT NULL,  
  description TEXT,  
  due_date DATETIME,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (course_id) REFERENCES courses(id)  
);
```

جدول تقديم المهام (Submissions)

SQL

```
CREATE TABLE submissions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  assignment_id INTEGER,  
  student_id INTEGER,  
  content TEXT,  
  submission_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (assignment_id) REFERENCES assignments(id),  
  FOREIGN KEY (student_id) REFERENCES users(id)  
);
```

جدول الدرجات (Grades)

SQL

```
CREATE TABLE grades (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  student_id INTEGER,  
  course_id INTEGER,  
  assignment_id INTEGER,  
  grade REAL,  
  graded_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (student_id) REFERENCES users(id),  
  FOREIGN KEY (course_id) REFERENCES courses(id),  
  FOREIGN KEY (assignment_id) REFERENCES assignments(id)  
);
```

Backend (Express.js) كود

app.js

JavaScript

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const authRoutes = require('./routes/auth');
const studentRoutes = require('./routes/students');
const professorRoutes = require('./routes/professors');
const adminRoutes = require('./routes/admin');
const { sequelize } = require('./config/database');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/professors', professorRoutes);
app.use('/api/admin', adminRoutes);

// Serve static files in production
if (process.env.NODE_ENV === 'production') {
  app.use(express.static(path.join(__dirname, '../frontend/build')));

  app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '../frontend/build', 'index.html'));
  });
}

// Database connection and server start
sequelize.sync().then(() => {
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
}).catch(err => {
  console.error('Unable to connect to the database:', err);
});
```

authController.js

JavaScript

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key';

exports.register = async (req, res) => {
```

```

try {
  const { username, password, email, full_name, role } = req.body;

  // Check if user already exists
  const existingUser = await User.findOne({ where: { username } });
  if (existingUser) {
    return res.status(400).json({ message: 'Username already exists' });
  }

  // Hash password
  const hashedPassword = await bcrypt.hash(password, 10);

  // Create user
  const user = await User.create({
    username,
    password: hashedPassword,
    email,
    full_name,
    role
  });

  // Generate JWT token
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    JWT_SECRET,
    { expiresIn: '24h' }
  );

  res.status(201).json({
    message: 'User created successfully',
    token,
    user: {
      id: user.id,
      username: user.username,
      email: user.email,
      full_name: user.full_name,
      role: user.role
    }
  });
} catch (error) {
  res.status(500).json({ message: 'Server error', error: error.message });
}
};

exports.login = async (req, res) => {
  try {
    const { username, password } = req.body;

    // Find user
    const user = await User.findOne({ where: { username } });
    if (!user) {
      return res.status(400).json({ message: 'Invalid credentials' });
    }

    // Check password
    const isValidPassword = await bcrypt.compare(password, user.password);
    if (!isValidPassword) {
      return res.status(400).json({ message: 'Invalid credentials' });
    }
  }
}

```

```

// Generate JWT token
const token = jwt.sign(
  { userId: user.id, role: user.role },
  JWT_SECRET,
  { expiresIn: '24h' }
);

res.json({
  message: 'Login successful',
  token,
  user: {
    id: user.id,
    username: user.username,
    email: user.email,
    full_name: user.full_name,
    role: user.role
  }
});
} catch (error) {
  res.status(500).json({ message: 'Server error', error: error.message });
}
};

```

middleware/auth.js

JavaScript

```

const jwt = require('jsonwebtoken');
const { User } = require('../models');

const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key';

const authenticateToken = async (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ message: 'Access token required' });
  }

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    const user = await User.findById(decoded.userId);

    if (!user) {
      return res.status(401).json({ message: 'Invalid token' });
    }

    req.user = user;
    next();
  } catch (error) {
    return res.status(403).json({ message: 'Invalid token' });
  }
};

const requireRole = (roles) => {
  return (req, res, next) => {
    if (!req.user) {

```

```

        return res.status(401).json({ message: 'Authentication required' });
    }

    if (!roles.includes(req.user.role)) {
        return res.status(403).json({ message: 'Insufficient permissions' });
    }

    next();
};

module.exports = { authenticateToken, requireRole };

```

كود Frontend (React.js)

App.js

JavaScript

```

import React, { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import Login from './pages/Login';
import StudentDashboard from './pages/StudentDashboard';
import ProfessorDashboard from './pages/ProfessorDashboard';
import AdminDashboard from './pages/AdminDashboard';
import './App.css';

function App() {
    return (
        <AuthProvider>
            <Router>
                <div className="App">
                    <Routes>
                        <Route path="/login" element={<Login />} />
                        <Route path="/student" element={<ProtectedRoute>StudentDashboard
/></ProtectedRoute>} />
                        <Route path="/professor" element=
{<ProtectedRoute>ProfessorDashboard /></ProtectedRoute>} />
                        <Route path="/admin" element={<ProtectedRoute>AdminDashboard />
</ProtectedRoute>} />
                        <Route path="/" element={<Navigate to="/login" replace />} />
                    </Routes>
                </div>
            </Router>
        </AuthProvider>
    );
}

function ProtectedRoute({ children }) {
    const { user, loading } = useAuth();

    if (loading) {
        return <div>Loading...</div>;
    }

    if (!user) {
        return <Navigate to="/login" replace />;
    }

```

```

    }

    return children;
  }

  export default App;

```

AuthContext.js

JavaScript

```

import React, { createContext, useContext, useState, useEffect } from 'react';
import { authService } from '../services/authService';

const AuthContext = createContext();

export function useAuth() {
  return useContext(AuthContext);
}

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      authService.getCurrentUser()
        .then(response => {
          setUser(response.data);
        })
        .catch(() => {
          localStorage.removeItem('token');
        })
        .finally(() => {
          setLoading(false);
        });
    } else {
      setLoading(false);
    }
  }, []);

  const login = async (credentials) => {
    try {
      const response = await authService.login(credentials);
      const { token, user } = response.data;
      localStorage.setItem('token', token);
      setUser(user);
      return { success: true };
    } catch (error) {
      return { success: false, error: error.response?.data?.message || 'Login failed' };
    }
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

```



```

};

const value = {
  user,
  login,
  logout,
  loading
};

return (
  <AuthContext.Provider value={value}>
    {children}
  </AuthContext.Provider>
);
}

```

authService.js

JavaScript

```

import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_URL,
});

// Add token to every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export const authService = {
  login: (credentials) => api.post('/auth/login', credentials),
  register: (userData) => api.post('/auth/register', userData),
  getCurrentUser: () => api.get('/auth/me')
};

```

Login.js

JavaScript

```

import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import './Login.css';

function Login() {
  const [credentials, setCredentials] = useState({
    username: '',
    password: ''
  });
}

```

```

const [error, setError] = useState('');
const [loading, setLoading] = useState(false);
const { login } = useAuth();
const navigate = useNavigate();

const handleSubmit = async (e) => {
  e.preventDefault();
  setLoading(true);
  setError('');

  const result = await login(credentials);

  if (result.success) {
    // Redirect based on user role
    const role = result.user.role;
    navigate(`/${role}`);
  } else {
    setError(result.error);
  }

  setLoading(false);
};

const handleChange = (e) => {
  setCredentials({
    ...credentials,
    [e.target.name]: e.target.value
  });
};

return (
  <div className="login-container">
    <div className="login-form">
      <h2>تسجيل الدخول للمنصة الجامعية</h2>
      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label htmlFor="username">اسم المستخدم</label>
          <input
            type="text"
            id="username"
            name="username"
            value={credentials.username}
            onChange={handleChange}
            required
          />
        </div>
        <div className="form-group">
          <label htmlFor="password">كلمة المرور</label>
          <input
            type="password"
            id="password"
            name="password"
            value={credentials.password}
            onChange={handleChange}
            required
          />
        </div>
        {error && <div className="error-message">{error}</div>}
        <button type="submit" disabled={loading}>
          {loading ? 'تحميل...' : 'تسجيل الدخول'}
        </button>
      </form>
    </div>
  </div>
);

```

```

        </button>
      </form>
      <div className="role-links">
        <p>هل أنت:</p>
        <Link to="/student">طالب</Link>
        <Link to="/professor">أستاذ</Link>
        <Link to="/admin">مدير</Link>
      </div>
    </div>
  </div>
);
}

export default Login;

```

خدمات الإشعارات

notifications.js

JavaScript

```

class NotificationService {
  constructor() {
    this.notifications = [];
  }

  addNotification(notification) {
    const newNotification = {
      id: Date.now(),
      ...notification,
      timestamp: new Date().toISOString(),
      read: false
    };
    this.notifications.unshift(newNotification);
    return newNotification;
  }

  getUnreadNotifications() {
    return this.notifications.filter(n => !n.read);
  }

  markAsRead(id) {
    const notification = this.notifications.find(n => n.id === id);
    if (notification) {
      notification.read = true;
    }
  }

  markAllAsRead() {
    this.notifications.forEach(n => n.read = true);
  }

  clearNotifications() {
    this.notifications = [];
  }
}

```

```
export default new NotificationService();
```

الحماية والأمان

حماية المسارات (Route Protection)

JavaScript

```
const ProtectedRoute = ({ children, requiredRoles }) => {
  const { user } = useAuth();
  const location = useLocation();

  if (!user) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (requiredRoles && !requiredRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return children;
};

// استخدام المسارات المحمية
<Route
  path="/professor/courses"
  element={
    <ProtectedRoute requiredRoles={['professor', 'admin']}>
      <ProfessorCourses />
    </ProtectedRoute>
  }
/>
```

التحقق من الصلاحيات

JavaScript

```
const RoleGuard = ({ roles, children }) => {
  const { user } = useAuth();

  if (!user || !roles.includes(user.role)) {
    return <div>ليس لديك صلاحية الوصول لهذه الصفحة</div>;
  }

  return children;
};
```

خطة التنفيذ

المرحلة 1: الإعدادات الأساسية (3-5 أيام)

إعداد المشروع وتثبيت المكتبات 1.

- إنشاء قاعدة البيانات والجداول 2.
- (Express.js) إعداد الخادم الأساسي 3.
- (JWT) إنشاء نظام المصادقة 4.

المرحلة 2: تطوير الواجهة الأمامية (5-7 أيام)

- تصميم واجهة تسجيل الدخول 1.
- إنشاء لوحات التحكم الأساسية 2.
- تصميم مكونات الطالب 3.
- تصميم مكونات الأستاذ 4.
- تصميم مكونات المدير 5.

(أيام 5-7) API المرحلة 3: تطوير

- للمصادقة APIs تطوير 1.
- للطلاب APIs تطوير 2.
- للأستاذ APIs تطوير 3.
- للمدير APIs تطوير 4.

المرحلة 4: الدمج والاختبار (3-5 أيام)

- API دمج الواجهة مع الـ 1.
- اختبار الوظائف الرئيسية 2.
- إضافة التحقق من الأخطاء 3.
- تحسين الأداء 4.

المرحلة 5: النشر والصيانة (2-3 أيام)

- نشر المشروع على خادم 1.
- إعداد قاعدة البيانات الإنتاجية 2.
- توثيق المشروع 3.
- إعدادات الصيانة 4.

ملفات التكوين

package.json (Backend)

JSON

```
{
  "name": "university-platform-backend",
  "version": "1.0.0",
  "description": "University platform backend",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "dev": "nodemon src/app.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.0",
    "sqlite3": "^5.1.6",
    "sequelize": "^6.32.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

package.json (Frontend)

JSON

```
{
  "name": "university-platform-frontend",
  "version": "1.0.0",
  "description": "University platform frontend",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.8.1",
    "axios": "^1.3.4",
    "tailwindcss": "^3.2.7"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "devDependencies": {
    "react-scripts": "5.0.1"
  }
}
```

مميزات إضافية مقترحة

1. نظام الإشعارات: إشعارات فورية عند وجود مهام جديدة أو رسائل.
2. تصدير التقارير: تصدير التقارير الأكاديمية للطلاب.
3. البحث المتقدم: البحث في المواد والطلاب بسهولة.
4. التصميم المتجاوب: دعم كامل للأجهزة المحمولة.
5. تسجيل النشاط: تسجيل جميع الأنشطة في النظام.
6. نظام النسخ الاحتياطي: نسخ احتياطي تلقائي للبيانات.

.هذا يوفر لك أساساً متكاملًا للمنصة. يمكنني مساعدتك في تطوير أي جزء محدد من المشروع أو توفير تفاصيل إضافية عن أي مكون